

Multimedia Information Retrieval

“Audio Information Retrieval System”

(2024-25)

Shri Prakash Yadav: D03000084, University Of Naples Federico II

Chapters

- 1. Introduction**
- 2. Dataset and Preprocessing**
- 3. Vectorisation & Indexing**
- 4. System Architecture**
- 5. Evaluation**
- 6. Streamlit App**
- 7. Conclusions and Future Work**

1. Introduction:

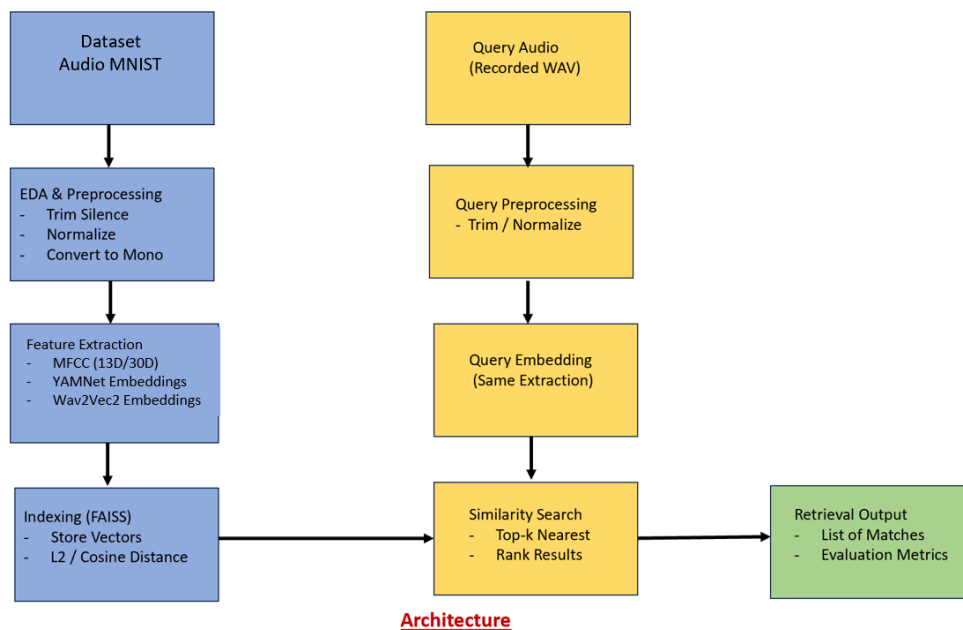
Information Retrieval is the field focused on finding relevant information in large collections of unorganised or semi organised data such as documents, web pages or multimedia. In early days, IR systems focused primarily on text retrieval, where the goal was to search and rank documents having similar keywords. Over the time, the scope of IR has increased significantly to include indexing, and retrieving information from different modalities such as audio, images, and video. This widened perspective is referred to as Multimedia Information Retrieval.

Audio Information Retrieval deals with representing and organizing audio data such as speech recordings, music, or other sounds so that users can search and retrieve audio according to their needs. Unlike text data, audio signals are continuous and high dimensional, require special techniques to extract numerical features that can be indexed efficiently.

In this project, I have explored various techniques to vectorise audio recordings into numeric representations. These representations are useful for the system to compare and rank audio samples based on their similarity to an audio query. The main objective of this work is to design and develop an audio retrieval system that allows users to query with an audio sample and retrieve the most relevant audio results.

2. System Architecture: -

The architecture of the retrieval system is illustrated below.



The pipeline is divided into two main parts: preparation of the AudioMNIST dataset embeddings and processing of query audio samples.

At the time of query, the same preprocessing and feature extraction steps are applied to the input signal. This query embedding is then matched against the index to retrieve the top-k most similar audio samples.

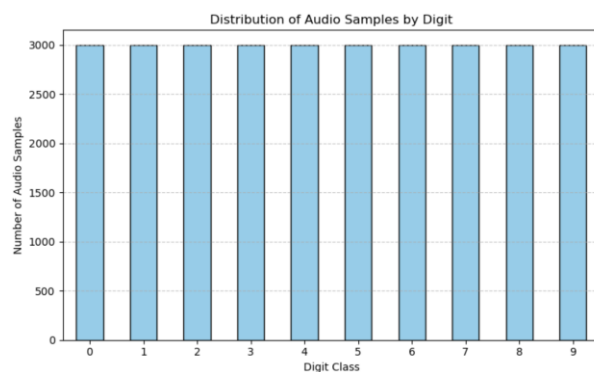
3. Dataset and Preprocessing:

The dataset I have chosen to work on is Audio MNIST. This is a clean dataset consisting of 30000 spoken audio samples. Each audio sample corresponds to a single digit from 0 to 9 recorded by people from different genders, accents, and age groups.

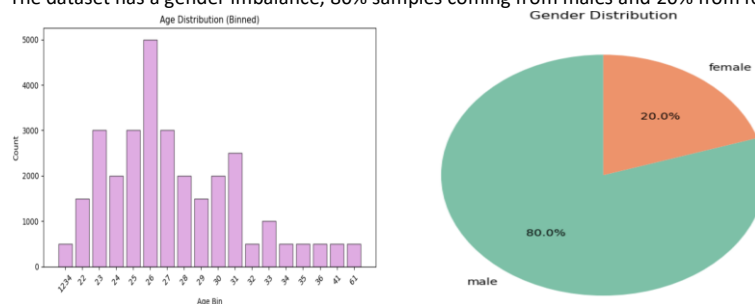
2.1 Exploratory Data Analysis (EDA):

I performed some Exploratory Data Analysis to understand the dataset and what kinds of problems were present. The EDA report is below.

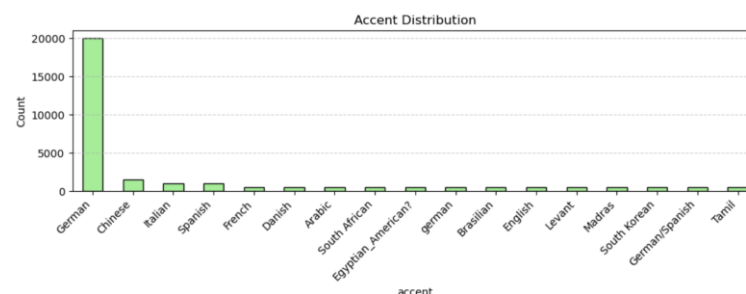
- The dataset contains a balanced distribution with 3000 audio samples per digit class.



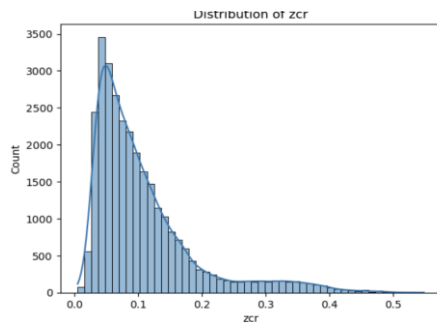
- Most samples are from people between ages 22–31, with an outlier age value of 1234. But, I didn't remove it as this is not a classification or regression project.
- The dataset has a gender imbalance, 80% samples coming from males and 20% from females.



- Also, the dataset has accent imbalance, with the vast majority of samples labelled as German accent, while all other accents have relatively few recordings. This will affect the performance when I will try to extract samples similar to my audio sample as I am Indian.

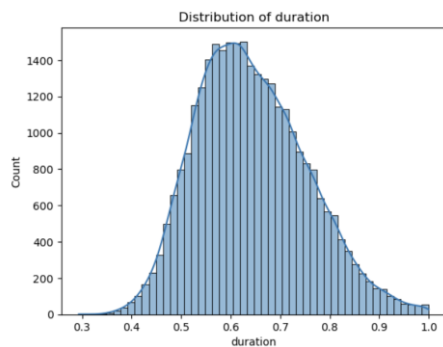


- **Zero Crossing Rate:** This Measures how often the signal waveform crosses the zero axis. Higher ZCR means more noisy or high-frequency, whereas lower ZCR means smoother waveforms.



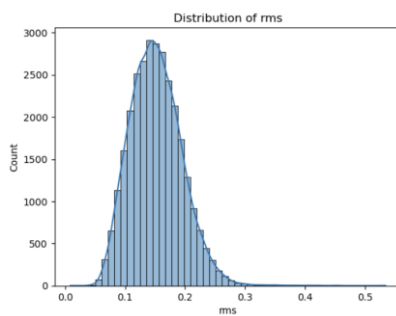
From the plot we can confirm that the majority of audio recordings have relatively low zero crossing rates, reflecting clean speech segments with minimal noise and high-frequency artifacts.

- **Duration:** Measured and visualized the distribution of audio durations to identify outliers.



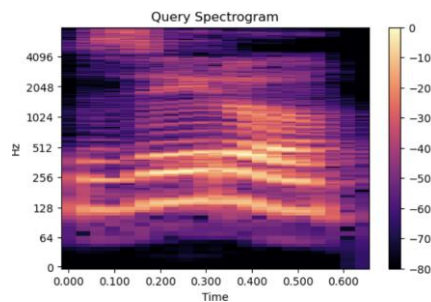
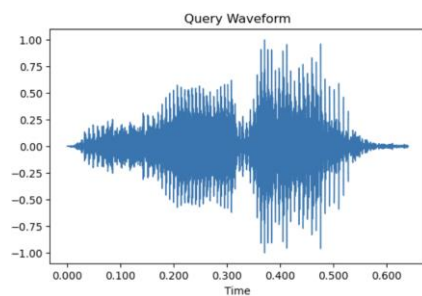
The distribution of audio durations indicates that most of the recordings are between 0.5 and 0.8 seconds long, with a clear peak around 0.65 seconds.

- **RMS:** Analysed RMS energy levels across the dataset to understand variations in loudness.



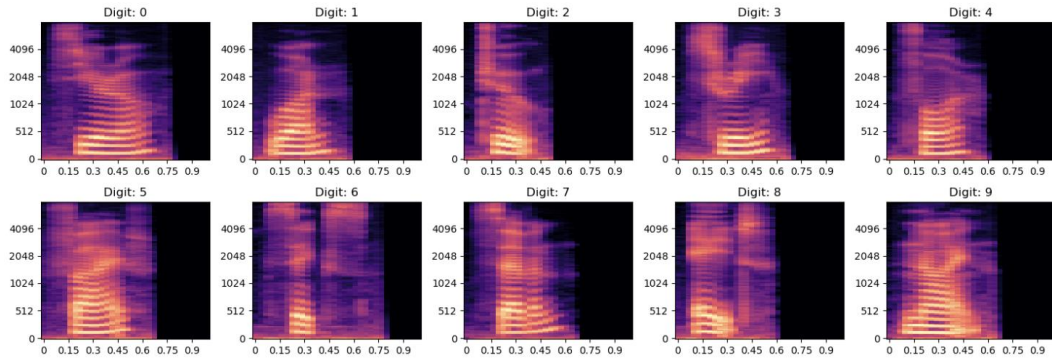
The distribution suggests that the most of the recordings have similar loudness, centered around 0.14. This narrow range reflects consistent recording conditions, very few samples having higher RMS values suggesting occasional louder signal, but these outliers are very in number.

- **Waveforms and Spectrograms:** Inspected waveforms and spectrograms to identify silence, clipping, and noise.



Waveforms and spectrograms representations confirmed that the dataset recordings are clean and consistently processed. Waveforms showed no evidence of clipping and clear amplitude variation corresponding to the spoken digits. Spectrograms exhibited well defined formant structures and minimal background noise.

Mel Spectrograms of First Sample for Each Digit



These plots show the differences in frequency content for each digit. Some digits appear to have similar frequency patterns such as 3, 4, and 5, as well as 1 and 7.

- For evaluation purpose, I recorded 10 audio samples (one for each digit) and below is the comparison between the recorded queries and dataset samples. The recorded queries are very few but they are necessary for retrieval performance evaluation.



We can see that there is a notable difference between query and dataset properties specially in ZCR. We will try to deal with this difference in the preprocessing part.

2.2 Preprocessing:

Overall, the AudioMNIST dataset has a clean and consistent collection of spoken digit recordings, reducing the need for extensive preprocessing. Exploratory data analysis confirmed that the majority of samples have low ZCR and consistent RMS energy, mean minimal background noise and uniform loudness. The durations of the recordings are also consistent, with most samples having 0.65 seconds length. Visualisation of waveforms, spectrograms, and MFCC representations also validate the quality of the dataset. As a result, no additional cleaning or normalization steps were applied prior to feature extraction.

Whereas, for the recorded queries, I performed the following preprocessing steps:

- Resampling:** Converted all recordings to a sampling rate of 16 kHz.
- Silence trimming:** Removed leading and trailing silence using amplitude thresholds.
- RMS normalization:** Scaled audio to a uniform loudness level.
- Duration standardization:** Padded or truncated signals to achieve a fixed length of one second.
- Noise reduction:** Applied light spectral denoising to reduce background noise.
- Augmentation:** To make my IR system more robust, I tried to apply augmentation to my audio collection, but the overall performance dropped. As a result, I dropped the augmentation part.

Finally, I have 30000 audio samples for the indexing and 10 audio signals for the query and evaluation.

4. Vectorisation & Indexing: -

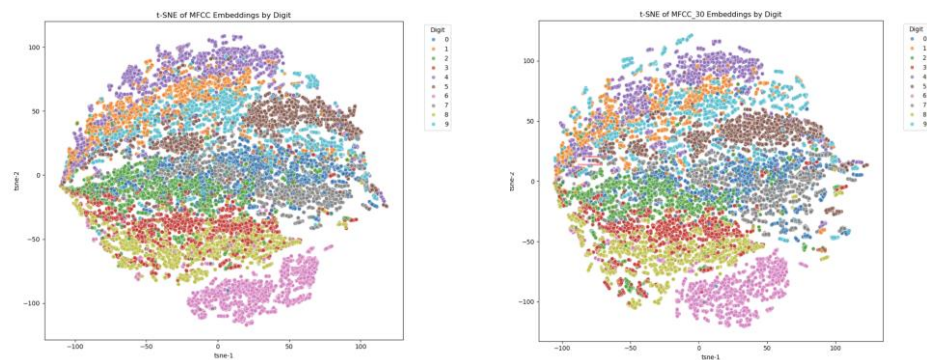
The Audio Retrieval System has two parts first one is the vectorisation of audio signals and the second one is the indexing.

3.1 Vectorisation: -

I explored multiple audio feature extraction techniques to represent the speech signals as numeric vectors. Following are the three technique/models, I tried:

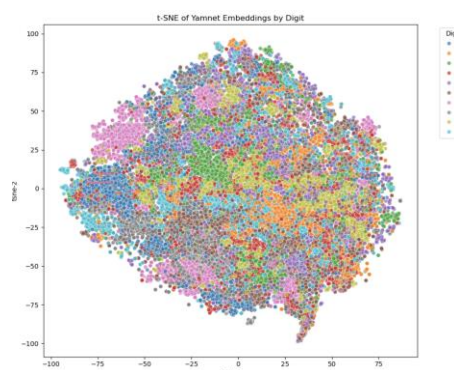
- **MFCC: -** Mel Frequency Cepstral Coefficients (MFCC) is a commonly used method to turn audio signals into sets of numbers that describe their main sound characteristics. It works by analysing how the energy of the audio is distributed across different frequencies in a way that reflects how humans hear sounds. In this project, I extracted MFCC features from each audio recording after resampling the signal to 16 kHz and standardizing its duration to one second. Recordings shorter than one second were padded with zeros, while longer recordings were truncated to ensure a fixed input length. I extracted MFCC vectors in two configurations 13 and 30 dimensions, by computing the coefficients for short frames of each recording and then averaging them over time to create a fixed-size vector for every audio file.

Below is the 2D scatter plot after dimensionality reduction using t-SNE for both 13D and 30D.



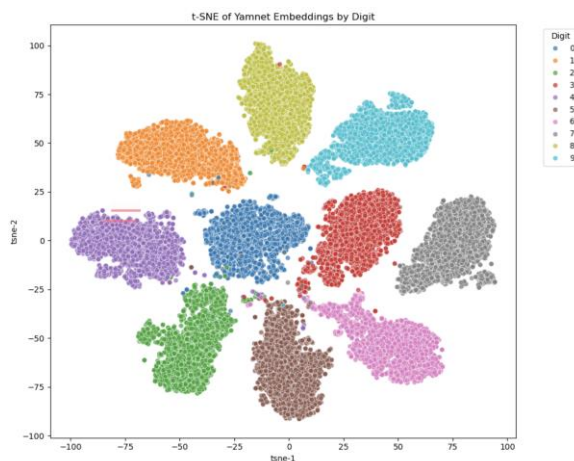
From the above plot, we can see that in 13D plot the classes are not well segregated but are better grouped than 30D MFCC featurisation.

- **YAMNet:-** YAMNet is a pretrained deep learning model developed by Google that is designed to recognize and classify a wide range of audio events. It is based on the MobileNetV1 architecture and is trained on AudioSet, a large dataset of diverse sounds. In this project, YAMNet was used as a feature extractor by passing each audio recording through the network and collecting the embeddings from a layer just before the classification layer. This layer gives a 1024-dimensional feature vector for each short time frame. I then average these vectors over time to get one fixed-size embedding for the whole audio file. Compared to MFCCs, YAMNet embeddings are better at describing general acoustic characteristics and can encode subtle differences between audio samples. This made them a useful baseline for evaluating how a more complex, pretrained model performs in an audio retrieval context.



The t-SNE visualization of YAMNet embeddings shows that the digits are spread throughout the embedding space without forming clear clusters. Some regions contain specific digits densely but there is significant overlap among different classes. This indicates that while YAMNet embeddings capture broad audio characteristics useful for general sound classification, they do not fully separate spoken digits.

- **Wav2Vec2:-** Wav2Vec2 is a deep learning model developed by Facebook AI for learning rich audio representations directly from raw waveforms. Unlike MFCC, that rely on handcrafted features, Wav2Vec2 is trained in a self-supervised way on large speech datasets to learn relevant patterns for recognizing and comparing spoken content. I used Wav2Vec2 as a feature extractor by passing each audio recording through the model and averaging the output from the last hidden layer across time to create a 768 length embedding. These embeddings capture high level speech feature, speaker characteristics, and overall acoustic patterns, making them especially effective for distinguishing between spoken digits. Compared to MFCCs and YAMNet, Wav2Vec2 provides a much more powerful and expressive representation of speech signals as illustrated in the below plot.



The t-SNE visualization of Wav2Vec2 embeddings shows that the digit classes form clearly separated and compact clusters, with very little overlap. Each color-coded region corresponds to one digit label, indicating that the embeddings successfully capture the distinctive acoustic features of each spoken digit.

The Wav2Vec2 embeddings show a much clearer separation between different digit classes compared to MFCC and YAMNet embeddings. Now I will create an index for each of the embeddings and evaluate which embedding model is best.

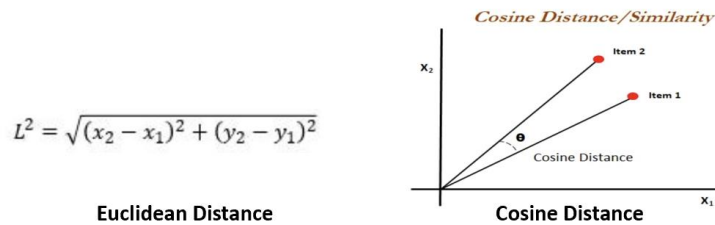
3.2 Indexing:-

After extracting feature embeddings for all audio recordings, the next step is to build an index that could store these vectors in a way that allows fast and accurate similarity search. Indexing is necessary because comparing a query embedding to every single sample in the dataset directly would be computationally expensive, especially with huge collection. I stored the embeddings obtained from MFCCs, YAMNet, and Wav2Vec2 in separate indexes.

- **FAISS:**
(Facebook AI Similarity Search) is an open-source library developed by Facebook AI Research designed to perform efficient similarity search and clustering of dense vectors, even at very large scales. Internally, FAISS organizes and stores feature embeddings as numeric arrays in a high-performance data structure, enabling fast retrieval of the nearest neighbors to a query vector using distance metrics such as L2 (Euclidean) distance or cosine similarity. In this project, I used a flat index with inner product (IP). After L2-normalization, IP behaves like cosine similarity.
- **Similarity Metric:-**

To measure how similar two embeddings are, the system used two types of distance metrics: Euclidean distance and cosine similarity. Euclidean distance (also called L2 distance) calculates the straight line distance between two points in the embedding space. This metric is effective for measuring the overall magnitude difference between vectors and was primarily used in the FAISS index to rank the nearest neighbours. In addition to this, cosine similarity was also experimented with, which measures

the cosine of the angle between vectors and focuses more on their direction rather than their scale. Cosine similarity is often preferred in cases where the embeddings are normalized, as it emphasizes how similar the vectors are rather than their absolute values. By comparing retrieval results with both distance measures, I tried to evaluate which approach produces more accurate matches for spoken digit queries.



5. Evaluation:-

For evaluation I recorded 10 audio queries, one for each numeric. Preprocessed them with the same pipeline as the AudioMNIST dataset audio signals. Then I extracted the embeddings of the test audio signals with each of the embedding models mentioned above. After that, I compared the retrieval accuracy of each embedding model.

For the evaluation, I used three following metrics:-

Precision@K:- It is the ratio of the relevant items among the top k results returned for each query. For example, if 7 out of the top 10 retrieved audio files belong to the correct class, the Precision@10 score is 0.7.

Mean Precision:- It is calculated by averaging the precision scores across all queries in the test set, providing an overall measure of how accurately the system retrieves relevant results on average.

Mean Reciprocal Rank (MRR):- It evaluates how highly the first relevant result appears in the ranked list of retrieved items. The reciprocal rank is 1.0 if the correct result is in the first position, 0.5 if it is in the second, and so on. Taking the mean of these values across all queries indicates how quickly users can expect to find the correct match in the retrieval output.

Below is the performance metrics table:-

Vectorisation	Precision @10 (Self Query / user Query)	Mean Precision (Self Query / user Query)	Mean Reciprocal Rank (Self Query / user Query)
1. MFCC 13D	0.9027 / 0.3100	0.9445 / 0.3125	0.9690 / 0.3125
2. MFCC 30D	0.9560 / 0.3000	0.9797 / 0.3143	0.9916 / 0.3167
3. Yamnet 1024D	0.6235 / 0.2600	0.7562 / 0.3145	0.8363 / 0.3510
4. Wav2Vec	0.9932 / 0.7000	0.9962 / 0.7333	0.9973 / 0.7333

We can see that Wav2Vec2 outperforms other embedding models by a notable margin.

After a few changes in the query audio preprocessing pipeline the Wav2Vec2 retrieval performance increased to



Evaluation Summary:

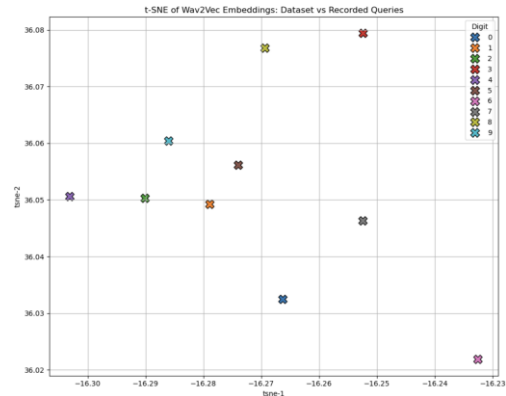
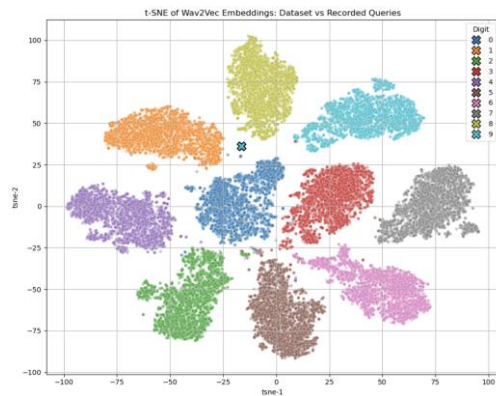
Mean Precision@10: 0.7100

Mean Average Precision (MAP): 0.7619

Mean Reciprocal Rank (MRR): 0.7643

After all the experiments the best performing system I get is the combination of Wav2Vec2 featurisation and the faiss cosine indexing.

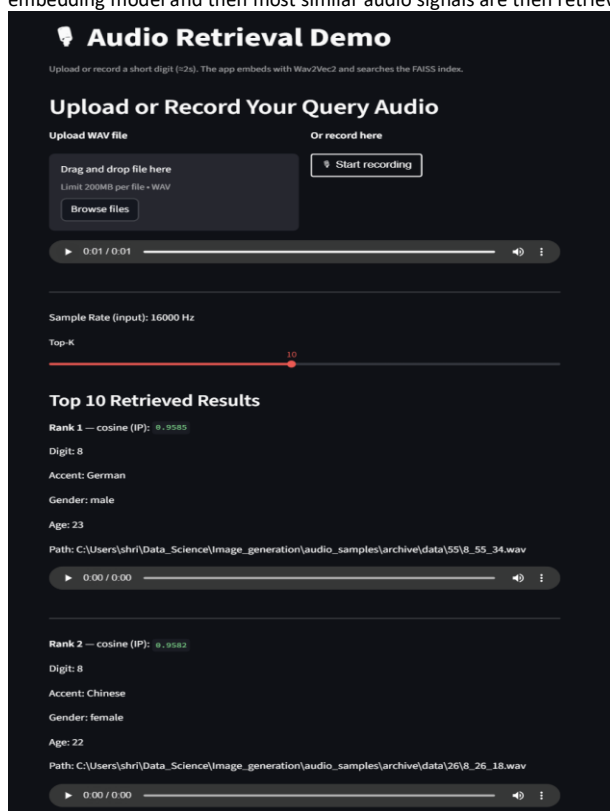
I tried to visualise why few audio signals fail to retrieve correct samples. I plotted the dataset collection and query embeddings next to each other.



I found out that when the recorded query embeddings are projected onto the same space, the queries appear shifted and distant from the main clusters. It can be due to domain differences between the original dataset recordings and the audio query samples, including possible variations in recording devices, surrounding noise, and speaker characteristics. As a result, the embeddings of the queries do not align perfectly with the dataset clusters, which sometimes causes retrieval errors despite the overall high performance of Wav2Vec2.

6. Streamlit App:-

Finally, I wrapped the whole project into a simple streamlit app. I created a end to end test time pipeline which consists of user query in form of audio recording/ audio upload. The recorded query is preprocessed in the same way, embedding is extracted using Wav2Vec2 embedding model and then most similar audio signals are then retrieved using the faiss cosine index.



7. Conclusion & Future Work

In conclusion, this project demonstrated the design and evaluation of an audio information retrieval system using multiple feature extraction techniques and FAISS-based indexing. Among all of the embedding models, Wav2Vec2 provided the most accurate retrieval performance, achieving clear separation between numerical digit classes. The system worked well with clean audio signals, but when experimented with recorded query the performance was not very good. The possible reason could be the accent difference.

The system could be extended by multimodal query, such as text queries or combined audio and text inputs, to enable richer search capabilities and improve user experience.